



南開大學  
Nankai University

# 《计算机图形学》实验报告

## (2025~2026 学年第一学期)

项目名称：手势交互式 3D 圣诞树

姓 名：李宗杭

学 号：2311451

专 业：软件工程

2026 年 1 月 3 日

# 目 录

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1 摘要</b>                                           | <b>2</b>  |
| <b>2 项目概述</b>                                         | <b>2</b>  |
| 2.1 目标与实现功能 . . . . .                                 | 2         |
| 2.2 技术栈 . . . . .                                     | 2         |
| <b>3 系统结构与渲染组织方式</b>                                  | <b>3</b>  |
| 3.1 模块划分 . . . . .                                    | 3         |
| 3.2 渲染入口与 Canvas 参数 . . . . .                         | 3         |
| <b>4 技术点</b>                                          | <b>4</b>  |
| 4.1 三维坐标系与变换链路 (World/Local/相机空间) . . . . .           | 4         |
| 4.2 围绕世界坐标轴旋转 (rotateOnWorldAxis) 与欧拉角/旋转累计 . . . . . | 5         |
| 4.3 程序化生成圣诞树 . . . . .                                | 5         |
| 4.4 球形空间“体积分布”的均匀散开 . . . . .                         | 6         |
| 4.5 螺旋装饰带 . . . . .                                   | 7         |
| 4.6 实例化渲染构建粒子系统 . . . . .                             | 7         |
| 4.7 帧率无关的“指数阻尼插值” . . . . .                           | 8         |
| 4.8 照片渲染 . . . . .                                    | 9         |
| 4.9 程序化纹理 (CanvasTexture) 用于装饰与点精灵 . . . . .          | 9         |
| 4.10 点精灵粒子系统 . . . . .                                | 10        |
| 4.11 主题化材质 . . . . .                                  | 11        |
| 4.12 光照、雾与环境贴图构建氛围 . . . . .                          | 12        |
| 4.13 后期处理 (Bloom+Vignette) . . . . .                  | 13        |
| 4.14 透明、混合与深度写入策略 . . . . .                           | 14        |
| 4.15 动画系统 . . . . .                                   | 14        |
| 4.16 交互量到变换量的映射 . . . . .                             | 15        |
| <b>5 效果展示</b>                                         | <b>16</b> |
| 5.1 TREE 模式 (经典主题) . . . . .                          | 16        |
| 5.2 FLOAT 模式 (冰雪主题) . . . . .                         | 16        |
| 5.3 FOCUS 模式 (浪漫主题) . . . . .                         | 17        |

## 1 摘要

本项目实现了一个运行于浏览器的交互式 3D 圣诞树场景，采用 Three.js 渲染内核并通过 React Three Fiber 组织渲染与状态。场景包含 1200+ 个装饰粒子与照片平面，支持“树形聚拢/球形漂浮/单张聚焦”三种模式切换，并提供三套主题（经典/冰雪/浪漫）在颜色、材质参数、粒子特效与氛围后期上联动变化。

报告主要包括坐标系与变换链路、程序化几何与分布采样、实例化渲染与性能优化、基于物理的材质与环境光照、纹理与点精灵（Point Sprite）粒子、透明与混合、后期处理 Bloom/Vignette、以及帧率无关的指数阻尼插值等。项目还涉及计算机视觉与手势识别的内容，此报告不详细展开，仅把其视作输入设备。

## 2 项目概述

### 2.1 目标与实现功能

- 在 WebGL 中构建“圣诞树 + 照片回忆”三维场景；
- 支持三种模式：
  - TREE（树形模式）：粒子与照片聚合成螺旋锥形圣诞树，自动缓慢旋转；
  - FLOAT（漂浮模式）：粒子与照片散开到球形空间，用户可控制旋转与缩放；
  - FOCUS（聚焦模式）：将某张照片固定到相机前方显示，可切换查看。
- 三种主题：圣诞/冰雪/浪漫；
- 视觉氛围：雾效、星空背景、Bloom 辉光、Vignette 暗角。

### 2.2 技术栈

- 语言与工程：TypeScript+React+Vite；
- 图形学：Three.js+@react-three/fiber+@react-three/drei；
- 后期：@react-three/postprocessing（Bloom, Vignette）；
- 交互输入：摄像头手势（MediaPipe Hands）+ 键鼠。

## 3 系统结构与渲染组织方式

### 3.1 模块划分

1. App.tsx: 全局状态机 (模式/主题/聚焦照片/输入开关)、照片列表管理、Canvas 创建;
2. components/Scene.tsx: Three 场景搭建 (灯光/环境/后期/特效), 以及“场景组”的旋转缩放控制;
3. components/TreeParticles.tsx: 实例化粒子系统 (InstancedMesh) 与插值动画;
4. components/PhotoCloud.tsx: 照片平面 (TextureLoader+Plane) 与聚焦时的坐标变换/面向相机;
5. utils/math.ts: 程序化分布采样 (螺旋锥形树、球形散开、螺旋装饰带);
6. utils/themeConfig.ts: 主题颜色与材质参数配置。

### 3.2 渲染入口与 Canvas 参数

项目通过 R3F 的 `<Canvas>` 将 React 组件树映射为 Three 场景树。

关键设置:

```
<Canvas shadows dpr={[1, 2]} camera={{ position: [0, 0, 20], fov: 45 }}>
  <React.Suspense fallback={null}>
    <Scene ... />
  </React.Suspense>
</Canvas>
```

- `dpr=[1,2]`: 根据设备像素比在清晰度与性能之间折中;
- `camera.fov=45`: 较常见的人眼透视视角, 避免广角畸变;
- `shadows`: 启用阴影 (场景内部分光源/网格设置了 `castShadow/receiveShadow`)。

## 4 技术点

### 4.1 三维坐标系与变换链路 (World/Local/相机空间)

#### 4.1.1 问题

在 Three.js 场景中，物体位置/旋转通常定义在局部坐标系，但交互（如“选择距离相机最近的照片”、“把照片固定到相机前方”）需要在世界坐标或与相机相关的坐标系中计算。需要解决：

- 如何把局部坐标转换到世界坐标以计算距离；
- 如何把相机前方的世界目标点，转换到某个 group 的局部坐标，以便把 mesh 放置在正确位置。

#### 4.1.2 实现思路

- 场景把所有粒子与照片放在 groupRef 下；当 groupRef 旋转/缩放时，子物体的局部坐标不变，但世界坐标会随 groupRef.matrixWorld 变化；
- “最近照片选择”：遍历照片的局部散开坐标 photo.scatterPos，用 matrixWorld 变换到世界坐标，再与 camera.position 计算距离；
- “聚焦照片固定相机前方”：
  1. 由相机四元数得到 forward 方向；
  2. 在世界空间算出  $worldTarget = camera.position + forward \times d$ ；
  3. 把 worldTarget 通过 group.worldToLocal() 转成 group 局部坐标，作为 mesh 的 targetPos；
  4. 旋转使用四元数 slerp，避免欧拉角万向锁。

#### 4.1.3 关键代码

选择最近照片：局部 -> 世界 -> 距离计算 (components/Scene.tsx)：

```
const selectClosestPhoto = () => {
  let minDist = Infinity;
  let closestId: string | null = null;
  const worldPos = new THREE.Vector3();

  photos.forEach(photo => {
    const localPos = photo.scatterPos;
    worldPos.copy(localPos);
    worldPos.applyMatrix4(groupRef.current!.matrixWorld); // local -> world

    const dist = worldPos.distanceTo(camera.position);
    if (dist < minDist) {
      minDist = dist;
      closestId = photo.id;
    }
  });

  if (closestId) setFocusedId(closestId);
};
```

聚焦照片：相机前方世界点 -> group 局部坐标 (components/PhotoCloud.tsx)：

```
const forward = new THREE.Vector3(0, 0, -1);
forward.applyQuaternion(camera.quaternion);

const worldTarget = camera.position.clone().add(forward.multiplyScalar(6));

targetPos.copy(worldTarget);
containerRef.current.worldToLocal(targetPos); // world -> local
```

面向相机：四元数插值 (components/PhotoCloud.tsx):

```
const dummy = new THREE.Object3D();
...
dummy.lookAt(localCamPos);
meshRef.current.quaternion.slerp(dummy.quaternion, dampRot);
```

## 4.2 围绕世界坐标轴旋转 (rotateOnWorldAxis) 与欧拉角/旋转累计

### 4.2.1 问题

在 FLOAT 模式下控制旋转时，希望旋转始终围绕“世界坐标轴”进行（类似绕地球自转轴），而不是围绕物体当前局部轴，这能带来更直观的操控感。

### 4.2.2 实现思路

- 对 groupRef 使用 rotateOnWorldAxis(axis, angle)，明确指定旋转轴为世界 X/Y；
- 将旋转速度 spinVelocityX/Y 随时间衰减（惯性），并通过 lerp 平滑收敛；
- 同时维护累计角度 rotationX/Y 便于状态追踪，并将其规范化到  $[-\pi, \pi]$  范围。

### 4.2.3 关键代码

(components/Scene.tsx):

```
const worldY = new THREE.Vector3(0, 1, 0);
const worldX = new THREE.Vector3(1, 0, 0);

groupRef.current.rotateOnWorldAxis(worldY, groupState.current.spinVelocityY * delta);
groupRef.current.rotateOnWorldAxis(worldX, groupState.current.spinVelocityX * delta);
```

角度规范化:

```
const TWO_PI = Math.PI * 2;
rotationY = rotationY - TWO_PI * Math.floor((rotationY + Math.PI) / TWO_PI);
```

## 4.3 程序化生成圣诞树

### 4.3.1 问题

如何在 3D 空间中生成“像圣诞树”的粒子分布，要求：

- 整体呈锥形：底部宽、顶部窄；
- 粒子不仅在表面，还要填充在体积内部；
- 视觉上带“环绕螺旋”的装饰感。

### 4.3.2 实现思路

1. 把粒子索引线性映射到高度  $y \in [-H/2, H/2]$ ;
2. 根据  $y$  线性映射该高度处最大半径  $R(y)$ ，从底部半径递减到顶部小半径;
3. 为了在圆锥体积内部均匀采样，半径使用  $\sqrt{u}$  做分布 ( $r = R \cdot u$  会导致粒子在中心更密，使用  $r = R \cdot \sqrt{u}$  可以让面积分布更均匀);
4. 使用角度  $angle = index * density + randomOffset$  形成螺旋趋势，并叠加随机扰动模拟树枝自然感。

### 4.3.3 关键代码

(utils/math.ts):

```
const y = MathUtils.mapLinear(index, 0, total, -height / 2, height / 2);
const maxRadiusAtHeight = MathUtils.mapLinear(y, -height / 2, height / 2, radius, 0.5);

const radialDistance = maxRadiusAtHeight * Math.sqrt(Math.random());
const angle = index * 0.5 + Math.random() * Math.PI * 0.3;

const x = Math.cos(angle) * radialDistance;
const z = Math.sin(angle) * radialDistance;

return new Vector3(
  x + (Math.random() - 0.5) * 0.4,
  y,
  z + (Math.random() - 0.5) * 0.4
);
```

## 4.4 球形空间“体积分布”的均匀散开

### 4.4.1 问题

漂浮模式要求粒子与照片散开到球形空间，并且视觉上“均匀充满球体体积”。若直接随机  $x, y, z$  再归一化，或随机半径  $r = Ru$ ，会导致密度偏差。

### 4.4.2 实现思路

在 generateScatterPosition 中使用经典的球体均匀采样：

- $\theta = 2\pi u$ ,  $\phi = \arccos(2v - 1)$  保证球面方向均匀;
- 半径用  $r = R \cdot \sqrt[3]{w}$  (立方根) 保证体积均匀。

### 4.4.3 关键代码

(utils/math.ts):

```
const u = Math.random();
const v = Math.random();
const theta = 2 * Math.PI * u;
const phi = Math.acos(2 * v - 1);
const r = Math.cbrt(Math.random()) * spread;

const x = r * Math.sin(phi) * Math.cos(theta);
const y = r * Math.sin(phi) * Math.sin(theta);
const z = r * Math.cos(phi);
```

## 4.5 螺旋装饰带

### 4.5.1 问题

除了树体粒子，本项目额外生成约 30% 粒子作为“缠绕在树外侧的装饰带”。核心是：给定参数  $t \in [0, 1]$ ，生成一条绕树旋转的空间曲线。

### 4.5.2 实现思路

- $y$  随  $t$  线性上升；
- 半径随高度递减（贴合锥体）并略放大 15% 形成外侧装饰带；
- 角度随  $t$  增长形成多圈螺旋。
  1. 由相机四元数得到 forward 方向；
  2. 在世界空间算出  $worldTarget = camera.position + forward \times d$ ；
  3. 把 worldTarget 通过 `group.worldToLocal()` 转成 group 局部坐标，作为 mesh 的 targetPos；
  4. 旋转使用四元数 `slerp`，避免欧拉角万向锁。

### 4.5.3 关键代码

(utils/math.ts):

```
const t = index / total;
const y = MathUtils.mapLinear(t, 0, 1, -height / 2, height / 2);

const treeRadiusAtHeight = MathUtils.mapLinear(y, -height / 2, height / 2, baseRadius, 0.5);
const spiralRadius = treeRadiusAtHeight * 1.15;

const angle = t * Math.PI * 12;
const x = Math.cos(angle) * spiralRadius;
const z = Math.sin(angle) * spiralRadius;
```

## 4.6 实例化渲染构建粒子系统

### 4.6.1 问题

粒子数量达到 1200+，若每个粒子一个 Mesh，会导致 draw call 数过多、CPU→GPU 提交开销大，帧率下降。



### 4.6.2 实现思路

采用 Three.js 的 InstancedMesh:

- 同一种几何体 + 同一种材质 -> 通过实例化一次性提交;
- 每个实例用 setMatrixAt(i, matrix) 指定位置/旋转/缩放;
- 颜色使用 setColorAt(i, color) 写入 instanceColor;
- 不同形状 (球/盒/锥/拐杖糖/小方片) 分为 5 个 instancedMesh, 各自维护实例数量。

### 4.6.3 关键代码

每帧更新实例矩阵 (components/TreeParticles.tsx):

```
particlesByShape.groups[shape].forEach(({ particle, index, instanceIndex }) => {
  dummy.position.copy(currentPositions.current[index]);
  dummy.rotation.x = ...;
  dummy.rotation.y = ...;
  dummy.scale.setScalar(particle.scale);

  dummy.updateMatrix();
  ref.current!.setMatrixAt(instanceIndex, dummy.matrix);
});

ref.current.instanceMatrix.needsUpdate = true;
```

主题切换时更新实例颜色 (components/TreeParticles.tsx):

```
ref.current!.setColorAt(instanceIndex, color);
...
ref.current.instanceColor.needsUpdate = true;
```

## 4.7 帧率无关的“指数阻尼插值”

### 4.7.1 问题

若使用固定比例的 lerp(current, target, alpha), 当帧率变化时动画快慢会不一致 (高帧率更快/更平滑, 低帧率更慢/更卡)。需要“帧率无关”的平滑公式。

### 4.7.2 实现思路

采用指数衰减 (阻尼) 形式:

$$\text{dampFactor} = 1 - e^{-k\Delta t}$$

然后用 current.lerp(target, dampFactor)。

- $k$  为速度系数 (可对每个粒子随机化, 增加有机感)
- $\Delta t$  为每帧时间间隔 delta

项目在粒子与照片两处都使用该方法, 使“树形 <-> 散开 <-> 聚焦”的过渡更丝滑。

### 4.7.3 关键代码

粒子 (components/TreeParticles.tsx):

```
const dampFactor = 1 - Math.exp(-physics.speed * delta);
currentPositions.current[index].lerp(target, dampFactor);
```

照片 (components/PhotoCloud.tsx):

```
const dampPos = 1 - Math.exp(-physics.speed * speedMultiplier * delta);
currentPos.current.lerp(targetPos, dampPos);
```

## 4.8 照片渲染

### 4.8.1 问题

如何把任意图片作为纹理贴到平面上，同时保持原始宽高比不变。

### 4.8.2 实现思路

- 使用 TextureLoader 加载照片纹理；
- 读取 texture.image.width/height 得到 aspect；
- 固定高度为 1，根据 aspect 调整平面宽度。

### 4.8.3 关键代码

(components/PhotoCloud.tsx):

```
const aspect = texture.image.width / texture.image.height;
if (aspect >= 1) {
  return { width: aspect, height: 1 };
} else {
  return { width: 1, height: 1 / aspect };
}
```

## 4.9 程序化纹理 (CanvasTexture) 用于装饰与点精灵

### 4.9.1 问题

为了减少外部资源依赖并快速迭代，本项目使用 Canvas 动态绘制纹理：

- 拐杖糖的红白条纹；
- 雪花粒子点精灵的径向渐变；
- 爱心粒子点精灵的形状纹理。

### 4.9.2 实现思路

- 用 `document.createElement('canvas')` 画图；
- 生成 `new THREE.CanvasTexture(canvas)`；
- 对贴图设置 `RepeatWrapping` 或用于 `pointsMaterial.map`。

### 4.9.3 关键代码

拐杖糖条纹纹理 (`components/TreeParticles.tsx`):

```
const canvas = document.createElement('canvas');
canvas.width = 128;
canvas.height = 128;
...
const texture = new THREE.CanvasTexture(canvas);
texture.wrapS = THREE.RepeatWrapping;
texture.wrapT = THREE.RepeatWrapping;
texture.repeat.set(2, 2);
```

雪花点精灵纹理 (`components/Scene.tsx`):

```
const grad = ctx.createRadialGradient(16, 16, 0, 16, 16, 16);
grad.addColorStop(0, '■ rgba(255,255,255,1)');
grad.addColorStop(1, '□ rgba(255,255,255,0)');
...
const texture = new THREE.CanvasTexture(canvas);
```

## 4.10 点精灵粒子系统

### 4.10.1 问题

冰雪主题与爱心主题需要大量飘落粒子。若用 `Mesh` (雪花模型/爱心模型) 逐个渲染, 开销大; 更高效的方式是 `Points` (点渲染) + 纹理模拟。

### 4.10.2 实现思路

- 用 `BufferGeometry` 存储所有点的 `position`, 以及每个粒子的 `velocity` (自定义 attribute);
- 每帧在 CPU 上更新 `position.y`, 并在越界后重置到顶部;
- 通过 `pointsMaterial+map` (点精灵纹理) 实现“雪花/爱心形状”的视觉效果;
- 透明混合时关闭 `depthWrite`, 减少深度写入导致的遮挡/闪烁。

### 4.10.3 关键代码

BufferGeometry attributes (components/Scene.tsx):

```
geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geometry.setAttribute('velocity', new THREE.BufferAttribute(velocities, 1));
```

每帧更新 +needsUpdate (components/Scene.tsx):

```
positions[i * 3 + 1] -= speed * velocities[i] * delta;
if (positions[i * 3 + 1] < -50) {
  positions[i * 3] = (Math.random() - 0.5) * 100;
  positions[i * 3 + 1] = 50 + Math.random() * 50;
  positions[i * 3 + 2] = (Math.random() - 0.5) * 60;
}
...
geometry.attributes.position.needsUpdate = true;
```

混合与深度设置 (components/Scene.tsx):

```
<pointsMaterial
  transparent
  depthWrite={false}
  blending={THREE.NormalBlending /*雪花*/}
/>

<pointsMaterial
  transparent
  depthWrite={false}
  blending={THREE.AdditiveBlending /*爱心*/}
  vertexColors
/>
```

## 4.11 主题化材质

### 4.11.1 问题

三种主题不只是换颜色，还要呈现不同“材质观感”：

- 经典：金属装饰（高金属度、低粗糙度）；
- 冰雪：更像玻璃/冰晶（高 transmission、清漆、彩虹色 iridescence）；
- 浪漫：更柔和、带发光与清漆。

#### 4.11.2 实现思路

- utils/themeConfig.ts 中把主题拆成两类配置：
  - ThemeColors: 颜色、背景、雾、灯光色
  - ThemeMaterials: PBR 材质参数
- 粒子使用 MeshPhysicalMaterial 统一基材, 并根据主题切换 roughness/metalness 等;
- 环境光照使用 Environment preset="city" 提供反射, 材质 envMapIntensity 提升金属质感。

#### 4.11.3 关键代码

主题材质参数 (utils/themeConfig.ts, 以冰雪为例):

```
export const SNOW_MATERIALS = {  
  goldMetalness: 0.8,  
  goldRoughness: 0.2,  
  goldEmissiveIntensity: 2.0,  
  useTransmission: true,  
  transmission: 0.95,  
  clearcoat: 1.0,  
  iridescence: 1.0,  
};
```

粒子基材 (components/TreeParticles.tsx):

```
new THREE.MeshPhysicalMaterial({  
  color: 0xffffffff,  
  roughness: themeMaterials.goldRoughness,  
  metalness: themeMaterials.goldMetalness,  
  envMapIntensity: 1.5,  
  clearcoat: themeMaterials.clearcoat,  
  transmission: themeMaterials.useTransmission ? themeMaterials.transmission : 0,  
  iridescence: themeMaterials.iridescence,  
});
```

### 4.12 光照、雾与环境贴图构建氛围

#### 4.12.1 问题

如何在一个相对简洁的模型场景里, 通过光照与雾营造空间感与节日氛围。

#### 4.12.2 实现思路

- AmbientLight 提供全局基础照度；
- PointLight（两盏）提供主题色的点光源，增强层次；
- SpotLight 从上方照射，形成树体的主光方向；
- FogExp2 给远处添加指数雾，增强深度感；
- Environment 提供环境反射。

#### 4.12.3 关键代码

(components/Scene.tsx):

```
<fogExp2 attach="fog" args={[`${...}`, 0.012]} />

<ambientLight intensity={0.2} />
<pointLight position={[10,10,10]} intensity={1} color={...} />
<pointLight position={[-10,5,10]} intensity={0.8} color={...} />
<spotLight position={[0,20,0]} angle={0.5} penumbra={1} intensity={2} castShadow />
<Environment preset="city" />
```

### 4.13 后期处理 (Bloom+Vignette)

#### 4.13.1 问题

在实时渲染中，后期可以显著提升观感：

- Bloom：高亮区域产生“溢光”，更梦幻；
- Vignette：边缘暗角，聚焦中心。

但 Bloom 过强会导致照片发白、细节丢失，需要控制阈值与强度。

#### 4.13.2 实现思路

- 使用 EffectComposer 管线；
- Bloom 使用较高 luminanceThreshold，避免照片整体参与 Bloom；
- 不同主题略微调整 intensity。

#### 4.13.3 关键代码

(components/Scene.tsx):

```
<EffectComposer enableNormalPass>
  <Bloom
    luminanceThreshold={0.6}
    mipmapBlur
    intensity={theme === ThemeType.SNOW ? 1.5 : 1.2}
    radius={0.4}
  />
  <Vignette eskil={false} offset={0.1} darkness={1.1} />
</EffectComposer>
```

#### 4.14 透明、混合与深度写入策略

##### 4.14.1 问题

透明物体（半透明照片、点精灵）在深度测试与混合下容易出现：

- 前后遮挡关系不稳定（排序问题）；
- 深度写入导致后续透明片段被错误剔除。

##### 4.14.2 实现思路

- 半透明点精灵关闭 depthWrite，保留 depthTest，减少自遮挡伪影；
- 对爱心使用 AdditiveBlending 营造发光叠加；
- 照片材质设置 transparent+ 适度 opacity。

##### 4.14.3 关键代码

点精灵：

```
<pointsMaterial transparent depthWrite={false} />
```

照片：

```
<meshBasicMaterial transparent opacity={0.9} />
```

#### 4.15 动画系统

##### 4.15.1 问题

若粒子完全静止，场景会显得死板；同时又不能让运动过强导致眩晕。需要可控的有机运动：

- 粒子轻微漂浮；
- 装饰物缓慢旋转；
- 树顶装饰呼吸式缩放。

### 4.15.2 实现思路

- 在 useFrame 中使用 clock.getElapsedTime() 得到连续时间  $t$ ;
- 用  $\sin/\cos$  产生平滑周期运动;
- 给每个粒子独立 phase (随机相位), 避免同步运动。

### 4.15.3 关键代码

粒子 idle 漂浮 (components/TreeParticles.tsx):

```
const idleY = Math.sin(t * 0.5 + physics.phase) * 0.15;
const idleX = Math.cos(t * 0.3 + physics.phase) * 0.15;
```

树顶装饰呼吸 (components/Scene.tsx):

```
starRef.current.rotation.y += delta * 0.3;
const scale = 1.0 + Math.sin(state.clock.elapsedTime * 2) * 0.1;
starRef.current.scale.setScalar(scale);
```

## 4.16 交互量到变换量的映射

### 4.16.1 问题

在漂浮模式下, 输入 (无论来源是手势还是鼠标) 通常是归一化屏幕坐标  $(x, y) \in [0, 1]$ 。需要把它转换成直观的旋转速度, 并加上“死区/最小速度/惯性”让体验稳定。

### 4.16.2 实现思路

- 令  $dx = x - 0.5, dy = y - 0.5$  表示相对屏幕中心偏移;
- 设置中心死区 DEADZONE, 在中心区域不旋转;
- 根据偏移距离计算强度 strength, 并映射到目标角速度;
- 用 lerp 平滑更新 spinVelocityX/Y;
- 缩放采用比例 distanceRatio 并用幂指数降低敏感度, 再 clamp。

### 4.16.3 关键代码

(components/Scene.tsx, 以手势为例, 鼠标逻辑同构):

```
const dx = handData.x - 0.5;
const dy = handData.y - 0.5;
const distanceFromCenter = Math.sqrt(dx * dx + dy * dy);

if (distanceFromCenter > DEADZONE) {
  const effectiveDistance = (distanceFromCenter - DEADZONE) / (0.5 - DEADZONE);
  const strength = Math.min(effectiveDistance, 1.0);
  targetVelY = dx * HAND_SENSITIVITY * strength + MIN_ROTATE_SPEED;
  targetVelX = dy * HAND_SENSITIVITY * strength;
}
```





### 5.3 FOCUS 模式（浪漫主题）

